



KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Yazılım Geliştirme I

C'den C++'a Geçiş

Hafta 1

C'den C++'a Geçiş

Doç. Dr. Tolga Berber

Bilgisayar Bilimleri · <https://avesis.ktu.edu.tr/tberber>

Öğrenme fazları:

1 • Ön-öğretim

2 • İçerik

3 • Katılım

4 • Değerlendirme

5 • İzleme

1 • Ön-öğretim Neden Önemli?

Birden çok kaynak dosyalı bir projede, derleyici tek bir dosyayı tek başına sorunsuz derlese bile en sonda şu hatayla karşılaşabilirsiniz:

```
1 // g++ main.cpp -> derleme BASARILI, ama baglama adiminda:  
2 undefined reference to topla(int, int)
```

- Derlemenin ve bağlamanın **iki ayrı aşama** olduğunu bilmezseniz bu hata anlamsız kalır
- C'nin gevşek tür kontrolünde uyumsuz bir dönüşüm çalışma zamanına kadar sessiz kalıp orada patlar
- Bu hafta “kod nasıl çalıştırılabilir hâle geliyor, hatalar hangi aşamada yakalanıyor?” sorusunu netleştireceksiniz

Bu Hafta

🎯 Öğrenme Hedefleri

Bu hafta sonunda şunları yapabileceksiniz:

- 1 Derleme + bağlama modeli; C++'ta tip güvenliği
- 2 namespace; `std::cin/std::cout` ile G/Ç
- 3 Referans (`int&`) ile C işaretçisini ayırt etmek
- 4 `auto` (tip çıkarımı) ve `const`
- 5 `.hpp/.cpp` ayrımı, namespace, CMake ile derleme

C'de derleme/işaretçi/struct, Python'da dinamik tipleme deneyiminiz var — bu hafta bu bilgiyi C++'a *taşıyorsunuz*.

BÖLÜM 1

Ünite 1 — Neden C++? C/Python'dan Köprü

Neden C++ namespace/iostream/Ref auto/const Kütüphaneyi Kurmak Özet

2. İçerik Aynı Model, Daha Sıkı Sözler

C++ de tıpkı C gibi **derlenen** bir dildir — Python'daki gibi kodunuzu satır satır yorumlayan bir program yoktur. Asıl fark çalışma modelinde değil, dilin size **verdiği sözlerde**.

C sürümü:

```
5 int main(void) {  
6     printf("Merhaba, C++!\n");  
7     return 0;  
8 }
```

C++ sürümü:

```
5 int main() {  
6     std::cout << "Merhaba, C++!\n";  
7     return 0;  
8 }
```

İkisi de aynı satırı basar — araç farklıdır: **printf** bir format dizgesi bekler, **std::cout <<** bir akışa değer gönderir.

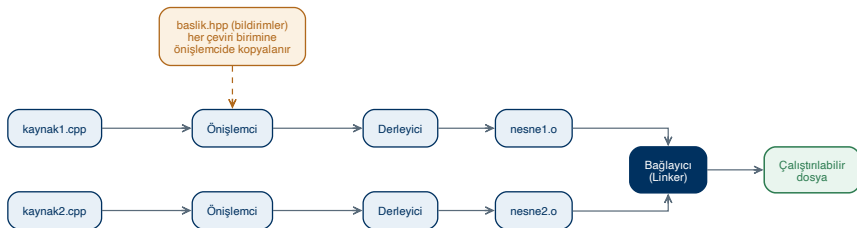
Derleme + Bağlama Modeli

Tanım: Derleme ve bağlama

Derleme, her `.cpp` dosyasını ayrı ayrı makine diline çevirip bir nesne dosyası (`.o`) üreten aşamadır. *Bağlama*, bu nesne dosyalarını birleştirip tek bir çalıştırılabilir dosya oluşturan sonraki aşamadır.

- Header dosyaları yalnızca **bildirim** taşır — önışlemci tarafından ilgili çeviri birimine olduğu gibi kopyalanır
- `a.cpp` + include ettiği header'lar → *çeviri birimi* — derleyicinin asıl gördüğü şey budur

Derleme + Bağlama Modeli



Her .cpp ayrı derlenir; bağlayıcı nesne dosyalarını tek çalıştırılabilirde birleştirir.

Analoji: Parça Fabrikası + Montaj Hattı

Analoji

Her `.cpp` dosyası kendi fabrikasında, diğer fabrikalardan habersiz biçimde kendi parçasını üretir — bu derleme aşamasıdır. Montaj hattı — bağlayıcı — bu parçaları alır, birbirine bağlar; tek bir ürün ortaya çıkar. Peki bir fabrika, hiç görmediği bir parçayla nasıl uyumlu üretim yapar? Elinde o parçanın *teknik şeması/krokisi* vardır — header dosyası tam olarak bu krokidir.

C++ Zihniyeti: Tip Güvenliği

- **Tip güvenliği**: C++ derleyicisi, C'nin görmezden geldiği birçok tip uyumsuzluğuna — `void*`, örtük dönüşüm — çok daha hazır itiraz eder
- Python'a kıyasla: derlenen/statik tipli C++'ta bir hata programı *hiç çalıştırmadan*, derleme anında yakalanır; Python'da aynı hata ancak o satır çalıştığında ortaya çıkar
- **RAII** (ilk bakış): bir kaynağın ömrünü bir nesnenin ömrüne bağlama fikri — otel oda kartı gibi, kart alındığında kaynak sizindir, kart iade edilince otomatik bırakılır (mekanizma Hafta 4'te)

CMake'e İlk Bakış

Birden çok `.cpp` dosyanızı elle derleyip doğru sırayla bağlamak hızla sıkıcı ve hataya açık hâle gelir.

Derleme sistemi

Derleme + bağlama adımlarını proje genelinde otomatikleştiren araç — bu ders boyunca kullanacağınız araç CMake'tir.

- Şimdilik yalnızca kavramsal: gerçek bir `CMakeLists.txt` dosyasını Ünite 4'te elinizle yazacaksınız

BÖLÜM 2

Ünite 2 — namespace, iostream, Referanslar

Neden C++ **namespace/iostream/Ref** `auto/const` Kütüphaneyi Kurmak Özet

2. İçerik namespace: İsim Çakışmasını Çözmek

Tanım: Ad alanı

Bir *ad alanı* (*namespace*), aynı isimli tanımların farklı “bölgelerde” çakışmadan var olmasını sağlayan bir isimlendirme kapsayıcısıdır. Dışarıdan erişim: `adAlanı::isim`.

Analoji

Ad alanını bir soyadına ya da şehir koduna benzetebilirsiniz: birden fazla “Ali” olabilir, ama soyadı ya da şehir hangi kişiden bahsedildiğini netleştirir. `std::cout` da böyledir — `cout`, `std` ad alanına “kayıtlıdır”.

using namespace std; Tuzağı

Dikkat

Bir header dosyasına `using namespace std;` yazarsanız, bu satır onu include eden `her .cpp` dosyasına kopyalanır — ve tam olarak ad alanlarının önlemeye çalıştığı isim çakışması riskini geri getirir.

- Tercih edilen yol: `std::` önekini her seferinde yazmak
- Ya da seçici: `using std::cout;` — yalnızca `.cpp` dosyalarında, header'larda `asla`

<iostream>: cin/cout

Analoji

Akış yönünü bir postane gönderim/teslim kutusuna benzetebilirsiniz: << oku sağa, hedefe doğru bakar (zarfı kutuya *gönderme*); >> oku sola, kaynaktan değişkene doğru bakar (zarfı *teslim alma*).

```
19 std::string isim;  
20 std::cout << "Adınız nedir? ";  
21 std::cin >> isim;  
22 std::cout << "Merhaba, " << isim << "!\n";
```

`cin >>` hedef değişkenin gerçek tipine bakar — C'nin `scanf("%d", &sayi)` format dizgesine kıyasla tip güvenlidir.

std::string: Bir Tadımlık

```
26 std::string ad = "Ada";  
27 std::string soyad = "LoveIace";  
28 std::string tamAd = ad + " " + soyad;  
29 std::cout << tamAd << " (" << tamAd.size() << " karakter)\n";
```

- + ile birleştirme — C'deki `strcat` zincirine gerek yok
- `.size()` — C'deki `strlen(buf)` yerine doğrudan üye fonksiyon çağırısı
- C'nin ham karakter dizisine kıyasla kendi belleğini kendisi yönetir

Referans: Lakap vs Yazılı Adres

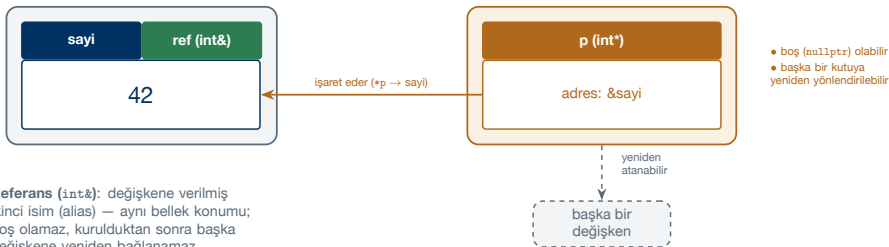
Tanım: Referans

Bir *referans* (T&), var olan bir değişkene verilen ikinci bir isimdir (takma ad) — boş olamaz, kurulduktan sonra başka bir değişkeni gösterecek biçimde **yeniden bağlanamaz**.

Analoji

Bir kişinin lakabı her zaman o kişinin kendisidir — boş olamaz, başka birine yeniden takılamaz. Yazılı bir ev adresi (işaretçi) ise boş bırakılabilir ve başka bir eve yeniden yönlendirilebilir; adresi okumak için “oraya gitmek” gerekir.

Referans vs İşaretçi



Referans (int&): değişkene verilmiş ikinci isim (alias) — aynı bellek konumu; boş olamaz, kurulduktan sonra başka değişkene yeniden bağlanamaz.

İşaretçi (int*): adres tutan ayrı bir değişken — null olabilir, sonradan başka bir kutuya yeniden yönlendirilebilir.

Referans: aynı kutuya ikinci isim, boş olamaz. **İşaretçi:** adres tutar, boş olabilir, yeniden yönlendirilebilir.

Referans Parametre Örneği

```
6 void ikiyeKatla(int& sayi) {  
7     sayi = sayi * 2;  
8 }  
9  
10 void referansOrnegiCalistir() {  
11     int sayi = 21;  
12     ikiyeKatla(sayi);  
13     std::cout << "İkiye katlanan sayi: " << sayi << '\n';  
14 }
```

- İmzadaki `&`: `sayi` çağırandaki gerçek değişkenin ikinci ismidir, kopya değildir
- Çağıran tarafta ne `&sayi` ne `*` yazarsınız — C'nin `ikiyeKatla(&sayi)` çağırısına kıyasla daha az gürültülü

BÖLÜM 3

Ünite 3 — auto, const, Modern C++ Tadımlığı

Neden C++ namespace/iostream/Ref **auto/const** Kütüphaneyi Kurmak Özet

2. İçerik auto: Tip Çıkarımı

Analoji

`auto`'yu bir otomatik vitesli araca benzetebilirsiniz: sürücü (siz) hâlâ nereye gideceğinize karar verirsiniz; `auto` yalnızca vites seçimini — değişkenin tipini — yol koşuluna (ifadenin sağ tarafına) bakarak sizin yerinize yapar.

```
13 auto a = 7;  
14 auto b = 35;  
15 auto toplam = a + b;  
16 std::cout << "Toplam: " << toplam << '\n';
```

Tip **derleme zamanında** belirlenir — Python'daki dinamik tipten temel farkı budur.

const: Değişmezlik Sözü

Analoji

`const`'u duvara çakılı bir tabelaya benzetebilirsiniz: bir kez asıldıktan sonra üzerinde oynanmaz.

```
6 void mesajYazdir(const std::string& mesaj) {  
7     std::cout << mesaj << '\n';  
8 }
```

İki fayda bir arada: `const std::string&` — kopya alınmaz (referans) **ve** gövde parametreyi değiştiremez (`const`).

Aralık Tabanlı for

```
22 int sayilar[] = {10, 20, 30, 40};  
23 for (int sayi : sayilar) {  
24     std::cout << sayi << '\n';  
25 }
```

- C'nin `for (int i = 0; i < 4; i++) sayilar[i]` yerine indeks yazmadan gezinme
- Bir-kayması (*off-by-one*) hatası riski baştan ortadan kalkar

BÖLÜM 4

Ünite 4 — Kütüphaneyi Kurmak

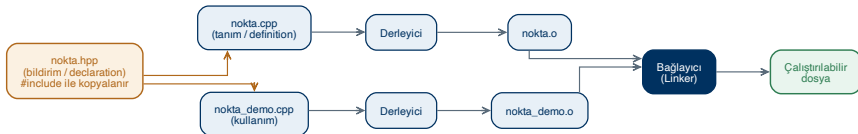
[Neden C++](#) [namespace/iostream/Ref](#) [auto/const](#) **Kütüphaneyi Kurmak** [Özet](#)

2. İçerik .hpp/.cpp Ayrımı: Nokta Yapısı

```
7 struct Nokta {  
8     double x;  
9     double y;  
10 };  
  
14 Nokta topla(const Nokta& a, const Nokta& b);  
15 void yazdir(const Nokta& n);
```

Bildirim (.hpp) her çeviri birimine kopyalanır; tanım (.cpp) yalnızca bir kez derlenir
— Ünite 1'in modelinin somutlaşmış hâli.

Header/Source Ayrımı



.hpp bildiriminden iki .cpp dosyasına include; her biri ayrı derlenip bağlamada birleşir.

#pragma once + namespace geometri

Analoji

`#pragma once`'u bir etkinlikte tek seferlik giriş damgasına benzetebilirsiniz — aynı davetliye elde ikinci kez damga vurulmaz.

- `#pragma once` olmasaydı: header içeriği aynı çeviri birimine iki kez kopyalanır, `struct` ikinci kez tanımlanmış olur → “yeniden tanımlama” hatası
- `namespace geometri`: **Nokta** adını isim çakışmasından korur — Hafta 3'te aynı kalıp `kitaplik` ad alanıyla, **Kitap** sınıfı için tekrarlanacak

CMake ile Derleme

```
1 cmake_minimum_required(VERSION 3.20)
2 project(hafta01_temel CXX)
3 set(CMAKE_CXX_STANDARD 20)
4 set(CMAKE_CXX_STANDARD_REQUIRED ON)
5 add_executable(nokta_demo nokta_demo.cpp nokta.cpp)
```

`CMakeLists.txt` projeyi ve C++20 standardını bildirir; böylece derleyici komutlarını elle yazmak yerine CMake'e bırakırsınız.

CMake: Derle, Çalıştır, Doğrula

Dene-Çalıştır

```
cmake -S . -B build  
./build/nokta_demo
```

→

```
cmake --build build
```

→

 Beklenen Çıktı

```
(1, 2) + (3, 4) = (4, 6)
```

BÖLÜM 5

Özet

Neden C++ namespace/iostream/Ref auto/const Kütüphaneyi Kurmak **Özet**

Özet

Özet

- C++ de C gibi derlenip bağlanır — fark çalışma modelinde değil, dilin verdiği sözlerde (tip güvenliği, RAI)
- `namespace` isim çakışmasını çözer; `<iostream>` tip güvenli giriş/çıkış sağlar; referanslar boş olamayan, yeniden bağlanamayan bir takma addır
- `auto` tipi derleme zamanında çıkarır; `const T&` kopyasız **ve** değişmez bir parametre verir
- `.hpp/.cpp` + `namespace` + CMake: dönem boyu büyüyecek kütüphanenizin ilk iskeleti

5 • İzleme Aktarım — Başka Nerede?

Bu haftanın kalıpları **Nokta** örneğine özgü değildir; birden çok dosyadan oluşan her C/C++ projesinde aynen işler:

- Derleme + bağlama modeli, çok dosyalı her projede geçerlidir — Ünite 4'te kendi **.hpp/.cpp** iskeletinizi kurarken bunu somut olarak yaşadınız
- **namespace** dönem boyu kendi kodunuzu düzenlemenin temel aracıdır; **const T&** ise büyük nesneleri kopyasız ve değişmez geçirmenin standart yoludur
- Hafta 3'te **Kitap** sınıfını tam olarak aynı kalıpla — **kitaplik** ad alanında, **.hpp/.cpp** + CMake ile — oluşturacaksınız: bugünkü iskelet Kitaplık projesinin ilk taş sırasıdır

Sırada Ne Var?

Bu hafta lab'da

`geometri::Nokta` örneğini *kendi elinizle* `.hpp/.cpp` olarak ayıracak, `CMakeLists.txt` yazacak ve derleyip çalıştıracaksınız.

- Ders notundaki **Kendinizi Sınavın** soruları her üniteyi kapsıyor: derleme + bağlama neden iki aşama, `cin >>` ile `scanf` farkı, `const T&`'nin iki faydası, `#pragma once` olmasaydı ne olurdu
- Gelecek hafta: fonksiyonları derinleştirip C++'ın **bellek modelini** — stack/heap, `new/delete` ve RAII'nin ilk somut örneğini — işleyeceksiniz